Содержание

1. Резюме для руководителя **1**

Как читать оценки . . . 1

2. Результаты панели **2**

Кто теряет баллы и на чём . . . 3

3. Частота проблем по проверкам **4**

Топ-4 и цена каждой ошибки . . . 5

4. Кейс FreshBite: до и после **5**

5. Самопроверка AppMOT: C, 62 из 100 **7**

6. Методология, точность и этика **8**

Известные режимы ложных срабатываний . . . 8

План v0.2 . . . 8

Этика публикации . . . 8

7. Выводы и следующие шаги **9**

1. Резюме для руководителя

Код теперь пишет каждый — а проверяет его по-прежнему почти никто. Пока генерация приложений ускоряется, зона создания становится всё более безопасной сама по себе: платформы выкатывают исправления и разумные дефолты. Но между «приложение собрано» и «приложению можно доверять» по-прежнему пусто. Этот отчёт — первый регулярный срез этой пустоты: что видит независимый чёрный ящик, когда смотрит на живые приложения так, как на них смотрел бы злоумышленник.

12 ноября 2026 года мы прогнали через сканер AppMOT v0.1 панель из 13 целей: девять крупных публичных платформ, «уязвимый двойник», собранный по лекалам типичной ночи с вайб-кодингом, его «вылеченная» версия и мы сами. Сканер делает 16 чёрно-ящичных проверок в четырёх категориях: секреты и утечки, заголовки и TLS, админки и доступ, живость и контент. Каждой цели присваивается балл 0–100 и класс A–D; одна находка критического уровня автоматически опускает потолок оценки до 44 баллов.

76,1	4/9	7/12	0 → 100
средний балл по рынку (9 платформ)	платформ с классом A	целей без полного набора заголовков	кейс «до/после» за часы работы

Главные выводы выпуска. Первый: даже у компаний с профессиональными командами безопасности гигиена веб-слоя проседает — больше половины панели теряет баллы на защитных заголовках, и только четыре платформы из девяти доходят до класса A. Второй: дистанция между «опасно» и «отлично» измеряется часами, а не месяцами, — наш собственный кейс показывает подъём с 0 до 100 баллов без единой строчки переписанного бизнес-кода, одной конфигурацией. Третий: автоматический сканер нуждается в честной дисциплине публикации — мы нашли у самих себя три режима ложных срабатываний и вынесли их в отдельный раздел вместе с планом уточнения; доверие к оценке важнее красивых цифр.

Как читать оценки

Правило	Значение
Стартовый балл	100
Провал или предупреждение проверки	минус штраф проверки (3–40 баллов)
Любая находка уровня critical	потолок оценки 44 балла (не выше C)
Любая находка уровня high	потолок оценки 74 балла (не выше B)
Класс A	85–100 баллов; класс B — 70–84; класс C — 45–69; класс D — ниже 45

Табл. 1. Правила оценки v0.1 (шкала с потолками)

Панель намеренно смешанная. Три собственных цели дают калибровку: «модель вайб-кода» показывает дно, «эталон» — потолок, самопроверка AppMOT — как выглядит обычный dev-инстанс до продакшена. Публичные платформы — это не рейтинг их инженерных команд, а нижний срез их веб-слоя, который видит внешний наблюдатель; этот срез и есть то, что видят и пользователи, и злоумышленники.

2. Результаты панели

Полная картина выпуска — ниже. Класс раскрашен по продуктовой шкале AppMOT: A — отлично, B — хорошо, C — требует доработки, D — опасно. Комментарий объясняет, из чего набран или потерян балл; спорные находки помечены как кандидаты на ложные срабатывания и разобраны в разделе 6.

Цель	Группа	Класс	Балл	Комментарий
FreshBite — уязвимый двойник	модель	D	0	дно шкалы: .env, .git и бэкап в открытом доступе, ключи в HTML, админка без защиты (Lovable)
FreshBite hardened — эталон	модель	A	100	потолок шкалы: TLS+HSTS, CSP, Basic-auth на админке, чистый клиентский код — 16 из 16
AppMOT — самопроверка (dev)	модель	C	62	http без TLS, нет CSP, dev source maps; сканер поймал «lorem ipsum» в списке наших же проверок
cloudflare.com	рынок	A	100	идеальный срез: все 16 проверок пройдены
anthropic.com	рынок	A	98	не хватает только Referrer-Policy
railway.app	рынок	A	92	часть защитных заголовков отсутствует
render.com	рынок	A	87	source maps и cookie-флаги; живой TLS и заголовки удерживают класс A

Цель	Группа	Класс	Балл	Комментарий
fly.io	рынок	B	80	доки-строки под «секрет», source maps, часть заголовков, мета-теги
vercel.com	рынок	B	74	cookie без флагов, CORS «*» на главной, /admin отдаёт 200
huggingface.co	рынок	C	66	catch-all роутинг: /wp-admin/ возвращает страницу модели; заголовков нет
netlify.com	рынок	D	44	заголовков нет; находка «секреты в коде» — вероятно, строки документации (см. §6)
mistral.ai	рынок	D	44	source maps открыты; «секрет» — строка из бандла Astro, кандидат на ложное срабатывание
openai.com	рынок	**н/д**	н/д	скан заблокирован WAF (403): в панель не входит

Табл. 2. Панель выпуска №1: 13 целей, 12 отсканировано, 1 заблокирован WAF

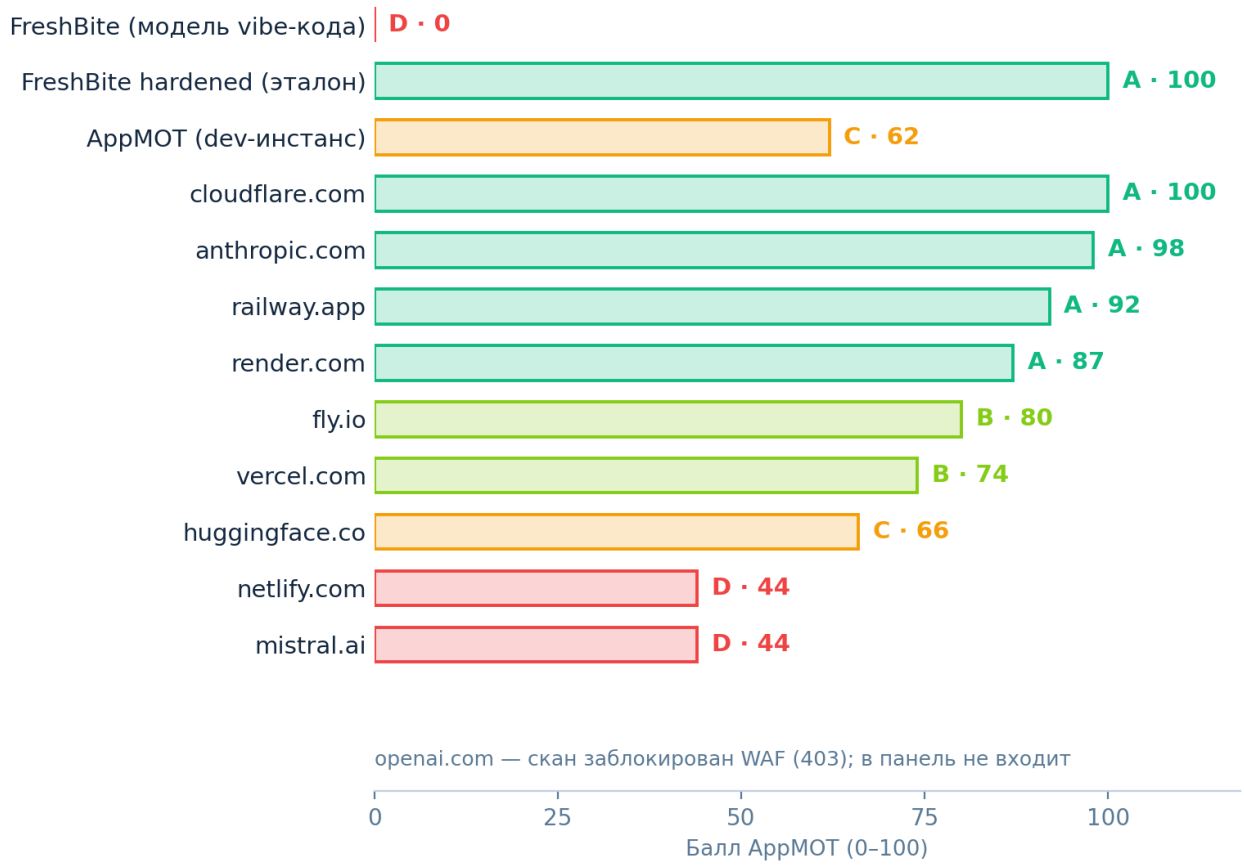


Рис. 1. Баллы панели по классам: модели (серые подписи) и публичные платформы; openai.com исключён — скан заблокирован WAF

Кто теряет баллы и на чём

Верхняя группа не удивляет: Cloudflare и Anthropic подходят к срезу с почти идеальной конфигурацией — это компании, для которых безопасность и есть продукт. Интереснее середина и низ. Netlify и Mistral получили класс D не потому, что у них дыры, а потому, что их маркетинговые страницы и бандлы содержат строки, похожие на секреты, а защитных заголовков нет. Мы помечаем эти случаи как кандидатов на ложное срабатывание, но убираем их из оценки только после ручной верификации — таков регламент выпуска.

Отдельного внимания заслуживает OpenAI: единственная цель панели, где сканер получил 403 на первом же запросе. Это честный результат бенчмарка, а не сбой: периметр WAF не отличил нашего робота от вредоносного. Для методологии это нижняя граница применимости чёрно-ящичных сканов: часть цели просто недоступна для наблюдения, и честнее вывести её из панели, чем рисовать «0 баллов за блокировку».

3. Частота проблем по проверкам

Если свести все 15 сработавших проверок по всем 12 отсканированным целям, картина получается такой:

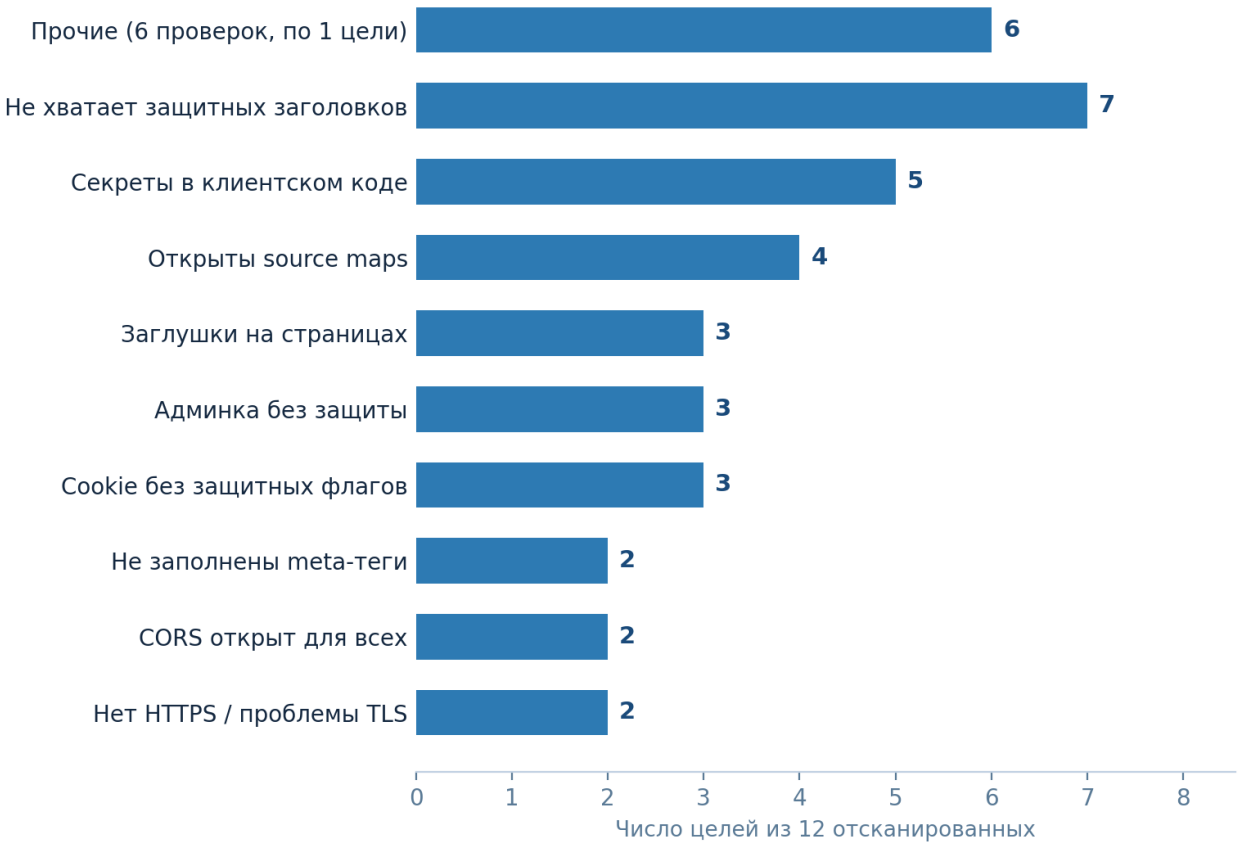


Рис. 2. Число целей (из 12), где сработала проверка; «прочие» — .env, .git, бэкапы, debug-эндпоинты, битые ссылки и ошибки в разметке (по одной цели)

Топ-4 и цена каждой ошибки

Защитные заголовки — 7 целей из 12. Самая частая и самая дешёвая проблема выпуска. CSP, X-Content-Type-Options, X-Frame-Options и Referrer-Policy ставятся одним блоком конфигурации за вечер, но их отсутствие открывает кликджекинг, MIME-sniffing и часть XSS-векторов. Ни одна модельная цель и меньше половины рыночных не закрыли этот список полностью.

Секреты в клиентском коде — 5 целей. Формально самая дорогая находка (штраф 40, потолок 44): всё, что видно браузеру, видно и злоумышленнику. В панели почти все случаи — строки из документации и маркетинга, а не живые ключи; но для vibe-coded приложений картина обратная — живые ключи в HTML там скорее правило, чем исключение. Проверка уже спасает: она единственная, которая у модели-двойника сработала вместе с .env и .git в категории «секреты».

Source maps — 4 цели. Публичные .map-файлы отдают исходники вместе с комментариями и путями. Не критично, но это бесплатная разведка для атакующего и маркер «сборка не для продакшена». Закрывается флагом сборки.

Cookie-флаги и админки — по 3 цели. Cookie без HttpOnly/Secure/SameSite отдают сессию скриптам и CSRF; админка, отвечающая 200 на стандартных путях без IP-ограничений, — приглашение к перебору. Обе проблемы лечатся конфигурацией и штатными средствами платформы.

Общий знаменатель топ-4: ни одна из проблем не требует переписывать приложение. Все четыре — вопрос конфигурации деплоя и гигиены сборки. Это и есть продуктовая идея AppMOT в одном абзаце: найти за минуту то, что чинится за вечер, пока это не нашёл кто-то другой.

4. Кейс FreshBite: до и после

Чтобы показать дистанцию между «опасно» и «отлично», мы построили один и тот же вымышленный продукт — ночную доставку еды FreshBite — в двух состояниях. «До» собран по лекалам типичной ночи с вайб-кодингом: работает, но .env и .git выложены в корень, ключи зашиты в HTML, админка открыта паролем без ограничений, на главной «coming soon» и «lorem ipsum». «После» — те же маршруты и тот же продукт, но прод-конфигурация: TLS с валидной цепочкой и HSTS, полный набор заголовков, Basic-auth перед админкой, чистый клиентский код.

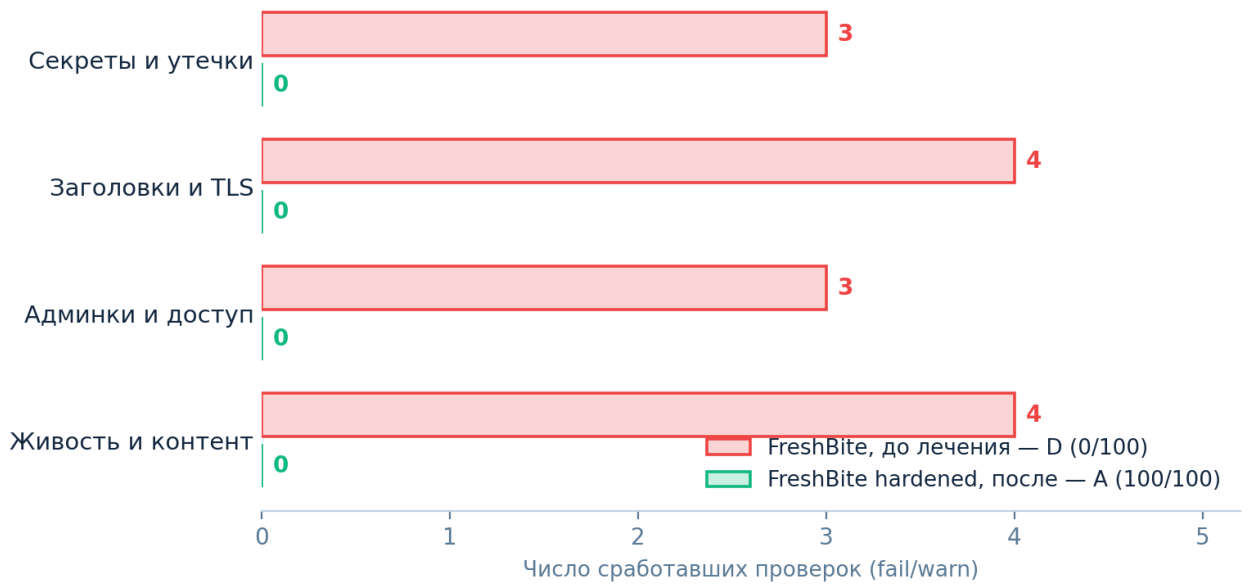


Рис. 3. Число сработавших проверок по категориям: FreshBite до лечения (D, 0/100) и FreshBite hardened (A, 100/100)

Проверка	FreshBite «до»	Эталон «после»
Файл .env доступен из сети	fail · критично (−40)	pass — 404
Папка .git доступна из сети	fail · критично (−35)	pass — 404
Секреты в клиентском коде	fail · критично (−40)	pass — чистый бандл
BaaS-база открыта на чтение	pass	pass
Source maps	pass	pass — не публикуем
Защитные заголовки	warn (−16)	pass — CSP, HSTS, nosniff, frame-ancestors
CORS для любого источника	warn (−8)	pass — заголовков CORS нет
Cookie без флагов	warn (−6)	pass — HttpOnly, Secure, SameSite=Strict
HTTPS / TLS	fail · high (−15)	pass — сертификат, 900 дней
Админка без защиты	warn (−15)	pass — 401, Basic-auth
Бэкапы по прямой ссылке	fail · критично (−40)	pass — 404
Debug-эндпоинты	warn (−8)	pass — 401, за авторизацией
Заглушки на страницах	warn (−4)	pass — живой контент
Ошибки в разметке	warn (−5)	pass
Битые внутренние ссылки	fail (−8)	pass — все разделы живы
Meta-теги	warn (−3)	pass — title и description

Табл. 3. Матрица «до/после»: все 16 проверок, штрафы и что именно изменилось

Итоговая дистанция: 0 против 100 баллов, D против A. Важно, что между состояниями нет ни одной строчки нового бизнес-кода: та же логика, те же маршруты, тот же продукт. Изменились сертификат, заголовки, флаги cookie, права на служебные пути и дисциплина сборки. По нашей оценке это вечер работы для одного человека или одна итерация автоматического фикса, который мы готовим для Pro-тарифа.

Для рынка это главный вывод выпуска: типичное vibe-coded приложение не обречено на класс D — оно просто ещё не прошло техосмотр. Порог входа в класс A находится не в инженерных знаниях, а в списке из шестнадцати пунктов, которые можно проверять автоматически. Смысл «техосмотра» ровно в этом: превратить невидимую ответственность в видимый чек-лист.

5. Самопроверка AppMOT: C, 62 из 100

Мы просканировали собственный лендинг — dev-инстанс, на котором работаем каждый день. Оценка: C, 62 балла. Разбор без скидок:

- **Нет HTTPS (−15).** dev-сервер слушает http: весь трафик ходит открытым текстом. В проде за прокси с TLS это уходит первым.
- **Нет защитных заголовков (−16).** Ни CSP, ни nosniff, ни frame-ancestors — тот же пункт, что у 7 целей из 12 в разделе 3.
- **Dev source maps (−3).** Карты исходников доступны в разработке; в проде Chromium их не публикует.
- **«lorem ipsum» (−4).** Наш любимый: на лендинге есть раздел «Заглушки и lorem ipsum» — список того, что мы ищем. Сканер честно нашёл заглушку в описании поиска заглушек.

Мы намеренно не подгоняли результат. C — честная оценка типичного dev-инстанса: три из четырёх замечаний закрываются прод-деплоем с TLS и заголовками ещё до первого пользователя, и это отчасти сама мораль выпуска — половина «плохих» оценок в дикой природе ставится приложениям, которые просто ещё не доехали до продакшена. Когда AppMOT переедет в прод за прокси с TLS, мы опубликуем повторную самопроверку в выпуске №2 — вместе с бейджем, как у всех.

Сканер, поймавший сам себя, — это не курьёз, а рабочий момент дисциплины: автоматическая оценка работает только тогда, когда ей не делают исключений. Мы не стали исключать и «lorem ipsum»: правильное уточнение правила — контекстный фильтр для описаний проверок, а не точечное исключение для себя любимых. Оно уже в плане v0.2.

6. Методология, точность и этика

Известные режимы ложных срабатываний

Сканер v0.1 — набор эвристик по внешним признакам, и у него есть честно задокументированные слепые зоны. В этом выпуске мы нашли три:

Цель	Что увидел сканер	Вероятная причина	Что делаем
netlify.com, mistral.ai	«секреты в клиентском коде»: строки вида sk-… в HTML и бандлах	примеры и фрагменты документации в маркетинге совпадают с сигнатурами живых ключей	энтропийный порог и контекст-фильтры в v0.2; до верификации находка остаётся в оценке
huggingface.co	«админка доступна»: /admin и /wp-admin/ отвечают 200	catch-all роутинг: любой путь возвращает страницу сущности, а не 404	детектор soft-404 в v0.2: сравнение тела ответа с телом главной и 404
AppMOT (мы сами)	«заглушки»: «lorem ipsum» на лендинге	упоминание искомого паттерна в описании самой проверки	исключать вхождения внутри описаний проверок и тегов code, а не руками

Табл. 4. Ложные срабатывания выпуска №1 и план уточнения

План v0.2

- Контекст-фильтр для сигнатур секретов: игнорировать вхождения внутри кодовых примеров, документации и описаний самих проверок.
- Энтропийный порог: строка должна выглядеть как сгенерированный ключ, а не как слово из маркетинга.
- Детектор soft-404: статус 200 с телом главной страницы не считается «страница существует».
- Публичный реестр находок с ручной верификацией по имени: спорные случаи проверяет человек до публикации.

Этика публикации

Правило выпуска простое: **агрегаты публикуем сразу, имена — только после ручной верификации и уведомления вендора.** Соревновательный рынок безопасности стимулирует публиковать чужие провалы громко и быстро; мы идём другим путём, потому что продукт продаёт доверие, а не страх. Найденная и подтверждённая проблема сообщается владельцу до публикации; отказ от публикации по запросу вендора фиксируется в реестре открыто. Сканер ходит только по публичным страницам, не эксплуатирует найденное, поддерживает щадящую нагрузку и opt-out по

письму — этот же регламент описан в юридическом разделе спецификации MVP.

7. Выводы и следующие шаги

Выпуск №1 фиксирует три вещи. Рынок: гигиена веб-слоя проседает даже у сильных игроков — больше половины панели без полного набора заголовков, и только четыре платформы из девяти в классе A. Продукт: дистанция между D и A — часы конфигурации, а не месяцы инженерии; кейс FreshBite прошёл её без единой новой строчки бизнес-кода. Дисциплина: автоматической оценке нужна ручная верификация по именам и открытый реестр собственных ошибок — иначе оценка сама превращается в вайб.

Что дальше. Выпуск №2 (декабрь 2026) расширит панель до 25–30 целей, добавит категорию «реальные vibe-coded приложения» с согласия их авторов и опубликует первый публичный срез уже с верифицированными находками. Параллельно выходит сканер v0.2 с контекст-фильтрами и детектором soft-404, а Pro-тариф получает автофикс топ-3 проблем из этого выпуска: заголовки, cookie-флаги и скрытие source maps — всё, что чинится конфигом, будет чиниться одним нажатием.

Если вы собираете приложения с помощью ИИ — пришлите нам URL. Первый техосмотр бесплатный, отчёт приходит минутой позже, а класс A — это вопрос одного вечера из списка выше. Проверим и вас, и нас: следующая самопроверка AppMOT выйдет в выпуске №2 — уже из продакшена.